

[Cynet Security Foundations](#)[Why Cynet](#) ▼[Platform](#) ▼[Partners](#) ▼[Resources](#) ▼[Company](#) ▼

Malware Anti-VM Techniques

Last updated on April 9, 2026

Written by: Itamar Medyoni

Getting the upper hand against common Anti-VM techniques malware authors use to evade the eyes of security professionals and malware researchers.

Introduction

Every day, Cynet's CyOps team receives files for inspection and analysis to determine the nature of these files and to extract important artifacts, which in turn assists our customers in achieving a quick forensic investigation and an efficient response.

The analysis and investigation process begins by dropping the file on a lab environment in order to gather the static and dynamic information and details. Later on, this will assist us in building the bigger picture, from understanding the type of malware to its persistency mechanisms, dropped files, living off the land techniques, further enumeration, and exploitation techniques.

On some occasions, I've observed Anti-VM Occurrences, which resulted only in some static findings, and in case the file has been packed and only unpacked in runtime, resulted in zero findings. This filled me with determination and my mind was set to solve the way this sneaky malware keeps evading me.

I sought the Internet for advice, and what I learnt was plenty. As such, today I will share with you, the readers, exactly how a malware author finds out if he is on a VM-based lab and how to overcome this hurdle with

We value your privacy

We use cookies to enhance your browsing experience, serve personalized ads or content, and analyze our traffic. By clicking "Accept All", you consent to our use of cookies.

[Customize](#) ▼[Reject All](#)[Accept All](#)



the infection stops and the attackers lose their virtual foothold on the host in case of a successful analysis.

[Why Cynet](#) ▼[Platform](#) ▼[Partners](#) ▼

In order to combat researchers, Malware Authors implement many different techniques to prevent the malware from performing its malicious action in a lab environment, or in some cases from running at all and, by that, preventing the researchers from inspecting any action the malware performs dynamically.

[Resources](#) ▼[Company](#) ▼

Are You For Real? Anti-VM Introduction

To run malware on a system inside your system, which is disconnected from any assets that should be protected, it is most common for malware researchers' labs to be built upon virtual machines. The other reason that VMs are so easy to use in such cases, is that they hold the ability to create a snapshot of the system and, by that, enable the researcher to save a copy of a current clean state, and roll back to it swiftly once the investigation is concluded or fails.

Malware authors are aware of this preference, and implement tests on their malware to inspect their running host before actually performing its malicious activities and close the process, or prevent it from running, if the test is positive.

Buckle-Up—A Real Life Scenario

In the following example, an analyst obtains a sample of the public malware—"Al-Khaser", which is a publicly-distributed malware. Upon static analysis, the file does not appear to show any malicious indicator, and when dynamically run on a Virtual Machine, the file closes.

To understand the reason for this file closing, moreover after having indication that it is malware and that it performs malicious activities, the only way to go is to read the instructions sent by the malware to the CPU line-by-line, and to attempt to determine which of the instructions was the one that shut down the process.

To perform such actions, running the file on an assembly debugger is a necessity. By using xDbg, and meticulous searches, I found the following code snippet. This assured me that the malware author had prepared for the likes of me, and that the infrastructure of the system was determined to be a lab and so, shut the process down:

We value your privacy

We use cookies to enhance your browsing experience, serve personalized ads or content, and analyze our traffic. By clicking "Accept All", you consent to our use of cookies.





```

mov dword ptr ds:[edi+8],ecx
push eax
mov dword ptr ds:[edi+C],edx
call al-khaser.9A4970
movups xmm1,xmmword ptr ss:[ebp-10]
add esp,C
xor ecx,ecx
movaps xmm0,xmm1
movq qword ptr ss:[ebp-6C],xmm1
psrldq xmm0,8
movd dword ptr ss:[ebp-64],xmm0 ; [ebp-64]:"A\xf"

```

Why Cynet ▾

Platform ▾

Partners ▾

Resources ▾

Company ▾

The code in the illustration utilizes native API calls to identify the brand of the CPU and if it includes the Hyper-V trait.

In this type of situation, the malware analyst is required to find a unique solution to masquerade as a lab as if it is not a VM, and manipulate system traits to deliberately fail this test.

The test above is just one of many that can be conducted by malware to determine the nature of the running system. In the next section, I will demonstrate and elaborate on each of the techniques with which I am familiar, and indicate ways how to circumvent them.

Under the Spotlight—CPU Interrogation

A. Hypervisor Brand by CPUID—CPUID is an instruction in the CPU architecture that provides the request processes for processor supplementary information. When malware retrieves the information from CPUID, by analyzing and comparing the value that is returned, it may understand in which type of system it runs. When running in a VM Architecture, the CPUID returns a known value depending on which vendor of the VM host you are using.

By supplying CPUID with a fixed value in EAX, and calling the CPUID, the ECX and EDX are next in line to populate with CPUID call:

We value your privacy

We use cookies to enhance your browsing experience, serve personalized ads or content, and analyze our traffic. By clicking "Accept All", you consent to our use of cookies.





```
asm {  
    xor eax, eax  
    mov  eax, 0x40000000  
    cpuid  
    cmp ecx, 0x4D566572  
    jne NopInstr  
    cmp edx, 0x65726177  
    jne NopInstr  
    mov  vb, 0x1  
NopInstr:  
    nop  
}  
cout << vb;  
return 0;  
}
```

[Why Cynet](#) ▼[Platform](#) ▼[Partners](#) ▼[Reasons to Buy](#) ▼[Company](#) ▼

The following Assembly code check whether the value of CPUID compares to a known value of the VMware architecture CPUID.

The code operates as follows:

1. Reset EAX and set its value to 0x40000000
2. Run the CPUID command which populates ECX and EDX with the Hyper Visor brand values
3. Compare the value of EDX and ECX to familiar values that reside in the VMware hosted machines
4. If the comparison is true, return **True** (1)
5. If the comparison is false, return **False** (0)

When running the compiled file in the memory of a non-virtual host, the output appears as follows:

We value your privacy

We use cookies to enhance your browsing experience, serve personalized ads or content, and analyze our traffic. By clicking "Accept All", you consent to our use of cookies.



[Why Cynet](#) ▼[Platform](#) ▼[Partners](#) ▼[Resources](#) ▼[Company](#) ▼

To bypass this method we need to change the value of the CPUID manually.

To do so, we first locate the VMX file of the machine, which holds within it the configuration of the machine.

 **Windows 10 x64.vmx**

Open the file in a text editor of your choice, and add the following lines at the end of the file:

```
cpuid.400000000.ecx="0000:0000:0000:0000:0000:0000:0000:0000"
```

```
cpuid.400000000.edx="0000:0000:0000:0000:0000:0000:0000:0000"
```

We value your privacy

We use cookies to enhance your browsing experience, serve personalized ads or content, and analyze our traffic. By clicking "Accept All", you consent to our use of cookies.





```
pciBridge4.pciSlotNumber = "21"
pciBridge5.pciSlotNumber = "22"
pciBridge6.pciSlotNumber = "23"
pciBridge7.pciSlotNumber = "24"
usb.pciSlotNumber = "32"
ethernet0.pciSlotNumber = "160"
sound.pciSlotNumber = "33"
ehci.pciSlotNumber = "34"
usb_xhci.pciSlotNumber = "192"
vmci0.pciSlotNumber = "35"
sata0.pciSlotNumber = "36"
nvme0.pciSlotNumber = "224"
svga.vramSize = "268435456"
vmotion.checkpointFBSIZE = "134217728"
vmotion.checkpointSVGAPrimarySize = "268435456"
ethernet0.generatedAddress = "00:0C:29:58:92:CA"
ethernet0.generatedAddressOffset = "0"
vmci0.id = "207131338"
monitor.phys_bits_used = "43"
cleanShutdown = "FALSE"
softPowerOff = "FALSE"
ethernet0.linkStatePropagation.enable = "TRUE"
sata0:1.startConnected = "TRUE"
toolsInstallManager.lastInstallError = "21004"
tools.remindInstall = "TRUE"
toolsInstallManager.updateCounter = "1"
cpuid.40000000.ecx="0000:0000:0000:0000:0000:0000:0000:0000"
cpuid.40000000.edx="0000:0000:0000:0000:0000:0000:0000:0000"
```

Why Cynet ▼

Platform ▼

Partners ▼

Resources ▼

Company ▼

Adding this setting replaces the default VMware CPUID.

When running the test again, we see the following results:

 Windows 10 x64 - VMware Workstation 15 Player (Non-commercial use only)

Player ▼ |   

 Select C:\Windows\System32\cmd.exe

```
Microsoft Windows [Version 10.0.18363.592]
(c) 2019 Microsoft Corporation. All rights reserved.
```

We value your privacy

We use cookies to enhance your browsing experience, serve personalized ads or content, and analyze our traffic. By clicking "Accept All", you consent to our use of cookies.



[Why Cynet](#) ▼[Platform](#) ▼[Partners](#) ▼[Resources](#) ▼[Company](#) ▼

```
using namespace std;

int main()
{
    bool isVm = false;
    __asm {
        xor eax, eax
        inc    eax
        cpuid
        bt     ecx, 0x1f
        jc     UnderVM
        NotUnderVM :
        jmp     NopInstr
        UnderVM :
        mov    isVm, 0x1
        NopInstr :
        nop
    }
    cout << isVm;
    return 0;
}
```

The following assembly code checks if the value of CPUID compares to a known value of VMware architectures CPUID:

The code operates as follows:

1. Reset EAX and increase its value one time
2. Run CPUID call
3. Perform a bit test to see if the 31st (represented in hex – 0x1f) bit is set or not
4. If the bit is set, return True (1)

We value your privacy

We use cookies to enhance your browsing experience, serve personalized ads or content, and analyze our traffic. By clicking "Accept All", you consent to our use of cookies.



[Why Cynet](#) ▼[Platform](#) ▼[Partners](#) ▼[Resources](#) ▼[Company](#) ▼

To bypass this method, we must change the value of the CPUID manually. To do so, first locate the VMX file of the machine that contains the configuration of the machine. Next, open the file in a text editor of your choice, and append the following lines in the end of the file:

 **Windows 10 x64.vmx**

```
cpuid.1.ecx="0—:—:—:—:—:—:—:—"
```



We value your privacy

We use cookies to enhance your browsing experience, serve personalized ads or content, and analyze our traffic. By clicking "Accept All", you consent to our use of cookies.





```
pciBridge6.pciSlotNumber = "23"
pciBridge7.pciSlotNumber = "24"
usb.pciSlotNumber = "32"
ethernet0.pciSlotNumber = "160"
sound.pciSlotNumber = "33"
ehci.pciSlotNumber = "34"
usb_xhci.pciSlotNumber = "192"
vmci0.pciSlotNumber = "35"
sata0.pciSlotNumber = "36"
nvme0.pciSlotNumber = "224"
svga.vramSize = "268435456"
vmotion.checkpointFBSize = "134217728"
vmotion.checkpointSVGAPrimarySize = "268435456"
ethernet0.generatedAddress = "00:0C:29:58:92:CA"
ethernet0.generatedAddressOffset = "0"
vmci0.id = "207131338"
monitor.phys_bits_used = "43"
cleanShutdown = "TRUE"
softPowerOff = "FALSE"
sata0:1.startConnected = "TRUE"
toolsInstallManager.lastInstallError = "21004"
tools.remindInstall = "TRUE"
toolsInstallManager.updateCounter = "1"
cpuid.400000000.ecx="0000:0000:0000:0000:0000:0000:0000:0000"
cpuid.400000000.edx="0000:0000:0000:0000:0000:0000:0000:0000"
usb_xhci:4.present = "TRUE"
usb_xhci:4.deviceType = "hid"
usb_xhci:4.port = "4"
usb_xhci:4.parent = "-1"
ethernet0.connectionType = "nat"
ethernet0.displayName = "VMnet0"
cpuid.1.ecx="0---:---:---:---:---:---:---:---"
```

[Why Cynet](#) ▼[Platform](#) ▼[Partners](#) ▼[Resources](#) ▼[Company](#) ▼

Adding this setting replaces the default VMware CPUID.

When running the test again we can see the following results:

We value your privacy

We use cookies to enhance your browsing experience, serve personalized ads or content, and analyze our traffic. By clicking "Accept All", you consent to our use of cookies.





```
C:\Users\ator\Downloads>
```

[Why Cynet](#) ▼[Platform](#) ▼[Partners](#) ▼[Resources](#) ▼[Company](#) ▼

From now on, whenever a malware attempts to test the CPL ID against VMware values, it fails to find it.

Knock Knock - Checking for Known Mac Addresses

Most virtual machine vendors include the feature to virtualize a custom NIC on your hosts.

If your lab is running with any VM native NIC, it is possible to enumerate its information using the following command line:

```
"wmic nic list"
```

This simple command uses the Windows Management Instrumentation tool to retrieve all information regarding all NICs, and among that information a MAC address is returned.

So, if you were using VMware, for example, MAC addresses will be presented with the following addresses:

```
00:05:69 (Vmware)
```

```
00:0C:29 (Vmware)
```

```
00:1C:14 (Vmware)
```

```
00:50:56 (Vmware)
```

Malware authors use this information to determine if this is a virtual or a physical machine.

To bypass this method, either stop using a VM-distributed NIC, or assign a MAC address manually in the adapter settings.

My name is Inigo Montoya – Process Name Enumeration to Detect a

We value your privacy

We use cookies to enhance your browsing experience, serve personalized ads or content, and analyze our traffic. By clicking "Accept All", you consent to our use of cookies.





```
print(re.sub('+', ' ', p))
```

Why Cynet ▾

Platform ▾

Partners ▾

This output is an example of these script outputs on a physical machine hosting a VM:

Resources ▾

Company ▾

```

C:\Windows\System32\cmd.exe
C:\Users\ator\PycharmProjects\untitled>py Check-Processes.py
\\WINDOWS\\Device\\Harddisk2\\Partition4 420 34986 7332 5252 936 5796 2203442290688 19864 8 5378048 23164 13 153 15 15
304 686244 0 2 1406250 2203432308736 10.0.19041 15998976 4 6420 \\r\\r\\n COVID vmnat.exe 7340 138 33750000 vmnat.exe Mic
rosoft Windows 10 Home|
\\WINDOWS\\Device\\Harddisk2\\Partition4 244140 5222046 6688 3260 936 4168 77922304 8688 8 3338240 7340 18 72 377 88 1
5518 9996596 0 4 16875000 72679424 10.0.19041 7032832 101311 140370638 \\r\\r\\n COVID vmnetdhcp.exe 1968 88 468750 vmnet
hcp.exe Microsoft Windows 10 Home|
\\WINDOWS\\Device\\Harddisk2\\Partition4 661 7480 7098 2300 936 2804 2203416375296 11424 8 2355200 10044 13 109 14 113
1 9511 0 2 312500 2203411881984 10.0.19041 10584064 0 0 \\r\\r\\n COVID vmware-usbarbitrator64.exe 26028 230 937500 vmware
usbarbitrator64.exe Microsoft Windows 10 Home|
\\WINDOWS\\Device\\Harddisk2\\Partition4 21444 103315 4722 2884 936 2984 2203429642240 11984 8 2953216 26028 14 137 17
138 975 12241 0 3 156250 2203423350784 10.0.19041 10600448 778 45437 \\r\\r\\n COVID vmware-authd.exe 26212 331 415000000
vmware-authd.exe Microsoft Windows 10 Home|
\\WINDOWS\\Device\\Harddisk2\\Partition4 178 17404 202897 49248 13696 74832 2212232626176 115776 4 50429952 9484 21 43
27 510 9690 5278994 1 15 25156250 2208082374656 10.0.19041 74903552 10900 5505680 \\r\\r\\n COVID vmplayer.exe 5528 590
42031250 vmplayer.exe Microsoft Windows 10 Home|
\\WINDOWS\\Device\\Harddisk2\\Partition4 1187 118492 4159 6212 720 6636 126009344 13224 8 6361088 1244 15 168 18 173 1
9 10806 1 2 312500 121413632 10.0.19041 11497472 1170 133067 \\r\\r\\n COVID vmware-unity-helper.exe 25716 262 625000 vmwa
re-unity-helper.exe Microsoft Windows 10 Home|
\\WINDOWS\\Device\\Harddisk2\\Partition4 1033 22620 7112 23144 25528 24328 191279104 19956 8 23699456 25716 22 294 23
96 13 14494 1 2 0 183414784 10.0.19041 16142336 35 2486 \\r\\r\\n COVID vmware-vmx.exe 12216 621 1634531250 vmware-vmx.exe
Microsoft Windows 10 Home|
C:\Users\ator\PycharmProjects\untitled>_

```

These are the known processes to be found in a VMware machine:

- Vmtoolsd.exe
- Vmwaretratr.exe
- Vmwareuser.exe
- Vmachtlp.exe

Look for the Key Master, Neo—Querying for Registry Keys

Another great way for an attacker to understand the type of machine he is currently in is by checking for the existence of registry keys.

We value your privacy

We use cookies to enhance your browsing experience, serve personalized ads or content, and analyze our traffic. By clicking "Accept All", you consent to our use of cookies.





```
using Microsoft.Win32;
using System;

namespace ConsoleApp1
{
    0 references
    class Program
    {
        0 references
        PREstatic void Main(string[] args)
        {
            Evasion Techniques

            string keyName = @"HKEY_LOCAL_MACHINE\Path\To\Known\Key";
            string valueName = "Expected Value Name";
            if (Registry.GetValue(keyName, valueName, null) == null)
            {
                Console.WriteLine("Doing nothing");
            }
            else
            {
                Console.WriteLine("Doing Malicious things");
            }
        }
    }
}
```

[Why Cynet](#) ▾[Platform](#) ▾[Partners](#) ▾[Resources](#) ▾[Company](#) ▾[NEXT](#) >[When a Sunburst Turns Supernova – A Recent S...](#)

Just Ask - Querying for Files

If we were thinking on a large scale, like going into the details and returning values of CPUID, it is clearer to understand the reason by performing a simple search for the file we believe to exist, to determine if the machine is a Virtual Machine or not.

Virtual Machines are known to hold within them special features and drivers that assist the operating system to function as if it was loaded by your motherboard (hence, “virtual” machines).

Malware Authors know to search for these simple files which are usually DLL files located in C:\windows\System32\Drivers\ , and yes, as you can guess by now, most of them actually contain a

We value your privacy

We use cookies to enhance your browsing experience, serve personalized ads or content, and analyze our traffic. By clicking "Accept All", you consent to our use of cookies.





Windows Script Host

[Why Cynet](#)[Platform](#)[Partners](#)

Doing Malicious Things

[Keep Reading](#)[Resources](#)[Company](#)

OK

[Attack Techniques](#)

Another way to show it through Python is to take a look at the following script which receives a list of common VirtualBox and VMWare known file paths and performs a check for each. If a file is found, exit the script, and do nothing. This illustrates how a chain of attack may be altered by asking for the existence of a file only:

Cynet's threat research report on MITRE ATT&CK T1036 (Masquerading), and how attackers rename malicious

```
import os.path
import sys

vm_files = ["C:\\windows\\System32\\Drivers\\Vmmouse.sys",
            "C:\\windows\\System32\\Drivers\\vm3dgl.dll",
            "C:\\windows\\System32\\Drivers\\vmdum.dll",
            "C:\\windows\\System32\\vboxoglpackspu.dll",
            "C:\\windows\\System32\\vboxoglpasssthroughspu.dll",
            "C:\\windows\\System32\\vboxservice.exe",
            "C:\\windows\\System32\\Drivers\\VmGuestLibJava.dll",
            "C:\\windows\\System32\\Drivers\\vmhgfs.dll",
            "C:\\windows\\System32\\Drivers\\vmtoolsd.dll",
            "C:\\windows\\System32\\Drivers\\VHIO\\vhook.dll",
            "C:\\windows\\System32\\Drivers\\VBoxMouse.sys",
            "C:\\windows\\System32\\Drivers\\VBoxGuest.sys",
            "C:\\windows\\System32\\Drivers\\VBoxSF.sys",
            "C:\\windows\\System32\\Drivers\\VBoxVideo.sys",
            "C:\\windows\\System32\\vboxoglcrutil.dll",
            "C:\\windows\\System32\\Drivers\\vmhgfs.dll",
            "C:\\windows\\System32\\vboxdisp.dll",
            "C:\\windows\\System32\\vboxhook.dll",
            "C:\\windows\\System32\\vboxmxnp.dll",
            "C:\\windows\\System32\\vboxogl.dll",
            "C:\\windows\\System32\\vboxtray.exe",
            "C:\\windows\\System32\\VBoxControl.exe",
            "C:\\windows\\System32\\Drivers\\vmGuestLib.dll",
            "C:\\windows\\System32\\vboxoglerrspu.dll",
            "C:\\windows\\System32\\vboxoglfeedbackspu.dll"]

for file in vm_files:
    if os.path.isfile(file):
        sys.exit()
```

[Attack Techniques](#)[Read More](#)[Attack Techniques](#)[Read More](#)[Top 6 Cyber Attack Prevention Strategies in 2025](#)[A cyber attack is an attempt to damage, disrupt, or gain unauthorized access to systems, networks, or technology-enabled ent...](#)[Quick Guide to MITRE ATT&CK: Matrices, Tactics, Techniques & More](#)[The MITRE ATT&CK framework is a knowledge-based repository of adversary tactics and techniques](#)[based...](#)

We value your privacy

We use cookies to enhance your browsing experience, serve personalized ads or content, and analyze our traffic. By clicking "Accept All", you consent to our use of cookies.





PLATFORM

The Platform

Endpoint Security

Network Security

SaaS & Cloud Security

Email Security

Mobile Protection

Identity Security

Security Automation

CyOps - 24x7 MDR

PARTNERS

Why Cynet 

Partner Program

Resources 

Managed Service Providers

Solution Providers &

Resellers

Partner Portal

Platform 

Cynet Resources

Company 

Threat Intelligence

Managed Service Providers

Managed Detection and
Response

Incident Response

Endpoint Security

Cybersecurity

Attack Techniques

AI Machine Learning

COMPANY

About us

Careers

News

Upcoming Events

Contact Us

Responsible Disclosure

WHY CYNET

2025 MITRE Evaluation
Results

Compare Cynet

Compliance and
Certifications

Packages

We value your privacy

We use cookies to enhance your browsing experience, serve personalized ads or content, and analyze our traffic. By clicking "Accept All", you consent to our use of cookies.





We value your privacy

We use cookies to enhance your browsing experience, serve personalized ads or content, and analyze our traffic. By clicking "Accept All", you consent to our use of cookies.

